
WHITE PAPER · WORK, ADOPTION & JUDGMENT

The Coordination Tax

Map work between tasks before claiming the savings

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Automating a task can push work into the handoffs, checks, and unusual cases around it; map and design that work before counting the savings.

Automating a task can shrink it while work between tasks (routing, translating, waiting, approving, reconciling, repairing) takes part of the saving. No source shows how often this happens, so this paper offers a test for one workflow. Trace real cases end to end and sort each handoff and check by what it produces. Then look for the new work around the tool, the handoffs, and the cases the tool cannot handle, and price that work at the seniority of the people who do it. Count freed capacity, the staff time the automation actually releases, only when the whole workflow shows it.

- 01** Trace real cases, not the process diagram: some of the hidden work sits in steps people added to make the official process function, and diagrams rarely show them.
- 02** Before cutting any step that exists to coordinate work, such as a review or handoff, ask what it produces; some reviews and handoff conversations are where defects get caught, and removing them makes failures more expensive.
- 03** Count freed capacity (the staff time the automation actually releases) only after the workflow has run long enough to settle, perhaps one to two quarters as a planning guide, or you may staff to savings that never arrived.

Where the saved time can go

Automation can cut the effort a task takes and still leave the work around the task untouched. It can even add to that work. Someone still has to route the job, reconcile the numbers, approve the output, explain it, and fix what breaks. This paper's claim is narrow. When a business case counts only the time saved on the task, the saving it reports can be real while the department keeps less of it, because some of the time moves into the work around the task. Leaders should map that work and measure the whole workflow before they claim a productivity gain or give the tool more independence. The paper does not say this happens in every workflow, and no source it

uses says how often it does. It offers a way to find out for one workflow.

One review I carried out makes the problem concrete. A team's drafting tool had made one task much faster, and the team was proud of it. The department's total output had not moved. Tracing real cases from request to delivery showed that some of the time had moved into work around the tool, including an approval step that existed only because the output was now machine-drafted. The task itself had shrunk. The work around it had grown.

In plain terms, that surrounding work is what this paper calls the coordination tax: the work between the tasks. Routing, translation, waiting, approval, reconciliation, explanation, repair. A task-level business case often has no line for it, because it is spread across many roles, buried in calendars and inboxes, and owned by no one in particular. The name is the paper's own. "Tax" only says that someone pays the cost whether or not a budget line shows it. Researchers have a formal definition worth borrowing. Malone and Crowston define coordination as the managing of dependencies among activities. Their examples include sharing a scarce resource, passing one activity's output to the next, and making sure that what is passed on is usable by whoever receives it. In this paper's reading, each is real work whether or not any ledger names it.[1]

The evidence for the paper's concern is partial, and it is better to say so at the start. The closest measurement comes from a study of AI chatbot adoption in Denmark, built on a late-2024 survey of about 25,000 workers in 11 occupations. Users reported saving about 3 percent of their work hours on average, and 85 percent of users said they spent the saved time on other job tasks. About 8 percent of users reported entirely new tasks created by the chatbots even where employers had no initiative, and roughly 17 percent where employers did. Saving time and spending it on other work is not a loss by itself, and the survey cannot say how much of that other work was new coordination. The new tasks ranged from generating content to checking the tool's output and fitting it into the workplace, and some reached workers who did not use the tools.[2] The study found no sizeable effect on earnings or recorded hours. These are survey answers, not time studies, and they cover chatbots, not the tool in the review above. The study shows that adoption can create new work. It does not show that the new work cancels the saving in any one workflow. That step is this paper's inference, and the review described below is the way to test it.

The stakes are in staffing. Business cases claim the hours a task no longer needs, budgets absorb the claim, and if the hours went elsewhere the department is left

staffed for savings it does not have. That is the paper's reasoning about how a task-level case can mislead, not an observed rate.

Why the tax stays invisible

Coordination work has three traits that keep it off the books, in this paper's reading. It is broken into fragments: a little routing here, a little reconciliation there, no piece big enough to name. It is spread out: each role carries a sliver, so no one person feels the total. It is absorbed: capable people take it on within their days without complaint, so it reads as busyness rather than cost. Work with those traits does not show up in time studies built around tasks, so an automation case built around tasks will not find it.

The pool it hides in is large. Cross, Rebele, and Grant report, from data they collected over two decades, that the time managers and employees spend in collaborative activities has risen by 50 percent or more, and that at many companies it takes around 80 percent of people's time.[3] That is the authors' own organizational data, and it measures collaboration time in general, not what automation adds. This paper's inference is only that the pool a task-based case never examines is big, so a small addition may pass unnoticed.

Automation can add work of its own. Someone has to check generated output against sources, put it into the format the next application needs, and explain it to the people who must decide what to trust. Malone and Crowston name the middle one as a dependency: what one activity produces has to be usable by the activity that receives it.[1] New approval steps can appear because the work is machine-made, and more odd cases can reach people at the edges of what the tool handles. How much work these add in a given workflow is an open question. That is why the paper asks for measurement and not a forecast.

Economists have a related result at a much larger scale. Brynjolfsson, Rock, and Syverson show that when firms adopt a general-purpose technology, measured productivity growth is understated at first, because resources go into complementary intangible investments that the statistics do not capture. They call it the productivity J-curve.[4] Their evidence is about firms and whole economies, not workflows, so it is an analogy for this paper's concern, not a measurement of it. The J-curve also treats the extra work as investment that can pay off later, which is one reason to measure after the work settles. The Danish study says its findings could suggest that some organizations are at the low point of that curve, while noting that many workers already report large time savings.[2]

Two software studies give further analogies. Sculley and colleagues argue that machine-learning systems are cheap to build and costly to maintain, with parts that

depend on each other in ways nobody planned.[5] Amrit and colleagues, mining code and communication records in a corporate environment, found clashes between how software was structured and how the team was organized in many projects.[6] Neither measures automation's side effects.

The coordination map

Tracing work across a process is not new. Lean value-stream mapping and process mining both do it. What this paper adds is the use. The trace becomes a test of an automation's business case: sort what it finds by what each step produces, find the new work the automation created, price that work at the seniority of the people doing it, and count freed capacity, the staff time the automation actually releases for other work, only after the whole workflow confirms it.

EXHIBIT 1

Six disciplines, in an order, to find where the saved time went

- 1 **Map work between tasks**
First: trace the whole path across people, applications, and queues, not one step.
- 2 **Tell useful coordination from waste**
Second: sort each link by what it produces; some interaction is worth keeping.
- 3 **Find the work around the tool**
Then, in any order: look for new translation between tool output and its users.
- 4 **Design handoffs deliberately**
Then, in any order: give every transfer a clear owner and acceptance test.
- 5 **Own the cases the tool cannot handle**
Then, in any order: count the cases outside the tool and the specialists they use.
- 6 **Measure the whole path**
Last: measure the whole path; tool usage and task time are supporting measures.

The map is a review of one workflow, run before a decision to scale. It studies the path rather than the task, and it puts the results into the same business case that proposed the automation. It has six disciplines, and they are not a checklist to run in any order. The first two come first. Tracing the path produces the links, each transfer, wait, or approval between two steps, and sorting the links by what each produces needs them in front of you. The next three are places to look once the links are sorted, and they can run together: the work around the tool, the handoffs that remain,

and the cases the tool cannot handle. The sixth comes last, because it judges whether the changes worked: it compares the whole path with the baseline from the first discipline once the changes from the middle three have settled. The sort in the second discipline has a group for work created by the change itself, and that group can only be filled after the middle three have found that work. Freed capacity is counted only at the sixth step. Nothing in the review needs new tooling. It needs observation, which is cheaper and rarer.

Map work between tasks

The first discipline traces real cases across people, applications, decisions, and queues, from the moment the demand arrives to the moment the outcome is delivered. Mark every link: each transfer, wait, approval, dependency, and backward loop. Trace the cases you observe, not the process diagram. The diagram shows the intended path. The tax can sit on the actual one, in steps people quietly added so the intended path would work in practice. Such steps are rarely in a document, and no owner may defend them.

Choose the workflow first. A workable one has a clear start and finish, such as a request arriving and an outcome being delivered, enough volume that a dozen recent cases exist, and an automation that is proposed or has recently shipped. Where the work has no clear finish, such as open-ended analysis, trace a sample of outputs from the request to what the person who received it did with it. Where no case records exist, ask the people who touched a case to rebuild it from their inboxes and calendars, and treat the result as a rough baseline. If cases cannot be traced at all, the whole-path measures in the sixth discipline still allow a before-and-after comparison, though without the links you will not know where the time went.

The evidence is a touch-and-queue ledger for each case: how many hands, how much waiting between hands, how much rework, which role carried each step, and why cases moved backward when they did. A dozen cases is a starting sample, not a proven threshold. Keep adding cases until new ones stop changing the map. The rule that follows: make no automation decision until the proposal can point at this map and say which links it removes, which it moves, and which it creates. An answer like "It makes the drafting step faster" covers one point on the map. The tax sits on the lines between the points.

In practice, pick twelve recent cases, some routine and some messy, and interview the people who touched each one about that case, not about the process. Ask what you needed before you could act, whom you waited on, and what you sent back. Plot the twelve journeys on one wall. Where the wall disagrees with the process diagram, you have found steps the organization added because the official path did not work.

Tell useful coordination from waste

The second discipline resists the efficiency reflex. Some interaction is how judgment forms, defects come to light, novices learn, and risk that crosses teams gets caught. It is the review that catches the mispriced contract, the handoff talk where the next team hears the case's history, the escalation that pulls a senior eye onto something unusual. Delete those links and you get a faster workflow that fails more expensively.

The test for each link is what it produces: a decision, a defect caught, knowledge passed on, a control, a relationship the work later depends on, or nothing but motion. Honest answers sort the links into four bins. Valuable: keep it, maybe strengthen it. Necessary but reducible: same job, lighter mechanism. Avoidable: a route or a rule can remove it. Created by the change itself: a cost of the automation, to be counted as one once the next three disciplines have found it. A program that treats every handoff as waste will delete checks that were catching mistakes.

Binning takes judgment. In an order-exception workflow I reviewed, a standing status meeting looked like a textbook deletion candidate. Traced against real cases, it turned out to be a place where pricing learned about inventory substitutions before customers did, so it was worth keeping. A recurring meeting that traces to no decision and no catch is the better candidate to retire. The bins come out of the cases, not the org chart.

Researchers are working on a formal test for the avoidable bin. Ju, in a 2026 preprint that has not been peer reviewed, proposes a test from distributed-systems theory for when coordination is required for correctness. Under the author's own breakdown of 65 standard workflows and 13,417 job tasks, 74 percent of the workflows and 42 percent of the tasks do not need it, which the author reads as up to 24 to 57 percent of coordination spending being unnecessary for correctness.[7] That is a model's output, and "not needed for correctness" is narrower than this paper's bins, which also count judgment, learning, and trust. Treat it as a hypothesis about kinds of work in general. The map tests one workflow.

Find the work around the tool

The third discipline hunts a form of the tax that automation itself creates: the work of translating between generated output and the applications and people that must use it. Users clean up formats. They re-key content into the official record. They check claims against sources. They note for later readers where the content came from. They prepare inputs the tool can handle. This work is easy to miss. It happens inside

the users' existing roles, in small amounts per case, at exactly the point the task measure does not cover.

The method is observation. Sit with real users. Time the work they do before and after the tool runs. Sort it into categories: preparing inputs, correcting output, translating formats, checking claims, explaining results. The findings convert into design work. If translation dominates, that points to interface or integration work. If checking dominates, it points to a record of where each claim came from, or to better data at the source. If explaining dominates, it points to output packaged differently for later readers. Work around the tool that cannot be designed away belongs in the business case as a permanent operating cost, priced at the seniority of whoever does it.

That seniority is likely to be high. Checking takes judgment, so it is likely to fall on experienced staff. If so, the automation's hidden payroll is drawn from the people the organization can least spare, while the visible saving was priced at an average salary. That is the paper's inference, and a ledger that records who does the minutes, not just how many, tests it. Where it holds, the ledger can flip a business case from positive to negative.

Design handoffs deliberately

The fourth discipline engineers the transfers that remain. A handoff is a small contract. It names an object with its evidence attached, a test the next team uses to accept it, an owner on each side, and the response expected. Automation can strain these contracts by changing volume and variety at machine speed. A sped-up earlier step can flood the next team's queue, which may lack the context, the capacity, or the authority to act on what arrives.

A handoff example from the same workflow: a faster automated classifier sent cases to a team that had no clear test for accepting them, and cases arrived missing information the resolvers needed. Faster intake alone cannot speed a workflow when the next team has to chase what is missing. A handoff contract addresses that: required context fields, an acceptance check at the mouth of the queue, and an intake rate matched to resolution capacity. None of it touches the classifier. The problem was the edge between the steps, not the automation.

The evidence is queue data at each transfer: age, acceptance rate, clarification requests, incomplete handoffs sent back, and receiving teams swinging between idle and overloaded. The design responses are unglamorous. Attach the context the next team needs. Set acceptance criteria so rejection is cheap and early. Slow the earlier step to what the next team can absorb. That last one is culturally hard, because

slowing an automated step feels like waste. When a queue grows faster than the team drawing from it, though, the extra work just sits there going stale, and that is not throughput. Measure what the next team can really absorb, not their stated capacity. A team that keeps up with the faster flow by triaging quicker and reading less deeply has traded a visible queue for lower quality that no one sees. Sampling the quality of the next team's decisions after a speed-up checks for this, and it belongs in the pilot design.

Own the cases the tool cannot handle

The fifth discipline counts the cases that fall outside what the automation can handle. Call them exceptions. When the routine path, the cases the automation handles cleanly, gets faster, more exceptions can reach people in absolute terms, and they are likely to fall to experienced staff.

A companion paper, *The Exception Path Is the Real Automated Workflow*, covers how to design that path and who owns it. This paper adds one line to the ledger: the specialist hours each exception consumes, including the search for an owner and the time spent rebuilding context, set against the hours the automation saves on routine cases. That ratio is the automation's margin on the workflow, and it can shrink as volume grows. Hold back on scale when exception demand approaches the capacity the routine path releases.

Measure the whole path

The sixth discipline moves the success measure from the task to the path. Value shows up, or does not, in end-to-end cycle time, total touches, outcome quality, capacity actually freed for other work, coverage expanded, decisions made faster, or risk reduced. Tool usage, task duration, and adoption measure activity at one point, while the claim being tested is about the whole path.

The method is a before-and-after comparison against the traced baseline, made once the changes from the middle three disciplines have settled: the same case types, the same path measures, and output quality sampled by people who know the work well enough to judge it. In the drafting review, task time had shrunk while the path had not improved, and the traced links showed where to look. Scale when the path improves and the remaining coordination is visible and owned. Redesign when the path is flat. A flat path under a celebrated task measure is a reason to trace, not proof that coordination ate the savings, since demand, staffing, or the mix of cases may also have changed. Even a short trace can turn up steps that the people

managing a workflow did not know existed. That is the practical argument for measuring rather than modeling when the decision is local.

Using the map to decide

The review's output fits on a wall: the traced path with links colored by bin, the work around the tool counted, exception costs attached, and the path-level baseline beside the proposal's claims. Decision-makers can then see what a task-level case never shows them: what the automation actually buys, at what coordination price, and where design work would change the answer.

EXHIBIT 2

The scale decision each discipline informs

Map work between tasks

Automate only after leaders can see whether the proposed change removes or relocates coordination.

Tell useful coordination from waste

Keep valuable coordination, redesign avoidable routing, and automate only the stable remainder.

Find the work around the tool

Include that work in the business case, and redesign the interface or process when it dominates.

Design handoffs deliberately

Throttle or reshape the earlier automated step when the receiving team cannot absorb the volume.

Own the cases the tool cannot handle

Constrain scale when recovery demand approaches the capacity saved on ordinary cases.

Measure the whole path

Scale when the workflow result improves and the remaining coordination is both visible and owned.

The map has a second use, sequencing. Most workflows hold several automation candidates, and the map shows which order pays best. Automating a step that feeds a queue already backed up just builds a bigger pile. Fixing the translation work first buys room for everything after it. A review that says "not yet, and here is what to do first" is the map earning its keep.

One costly error the map can prevent is declaring capacity freed before the path confirms it. Business cases claim the analyst hours the task no longer needs.

Budgets absorb the claim. If the hours went to work around the tool and to exceptions nobody counted, the department is staffed for savings that did not arrive. Counting freed capacity at the path level, after the coordination has settled, avoids that.

Objections and limits

The first limit is the evidence. No source used here shows how often an automation's savings move into surrounding work, or how large the shift is. The Danish survey shows that new work follows adoption. The J-curve and the software studies are analogies. The drafting review is one account, and the order-exception review is another. This paper's claim is therefore a test, not a forecast. If the whole path improves, the review confirms the business case at the planning estimate described below. If it does not, the review shows where to look.

Coordination is not inherently waste. Complex work needs consultation and judgment across teams. The tax is coordination whose purpose is unclear, whose route is avoidable, or whose cost was shifted out of view. Redesign should preserve necessary collaboration.

Nor is the map an argument against ambitious automation. Its verdicts can run the other way. A workflow whose coordination is mostly avoidable routing and re-keying is a candidate for deeper automation than anyone proposed, because the map shows the interaction would simplify rather than strain.

A third objection is speed: mapping slows an automation program while competitors move faster. The review is not free. Tracing means interviews and observation sessions with the same people the automation was meant to relieve, so ask for the shortest sample that changes the decision. A small, reversible automation on a low-volume workflow may not justify a review at all, so match the effort to what is at stake. For a workflow many people depend on, a program that skips the review ships more automations per quarter and may take on unpriced work in each, which falls on the same experienced people. That is this paper's reasoning, not a measured result. A review of about two weeks per workflow is the price of knowing what the automation actually does.

What to do next

The decision for leaders is what counts as done. An automation is complete when the path improves, the remaining coordination has owners, and the freed capacity survives a path-level audit. Leaders do not need to run the reviews themselves. Holding that definition against vendors, against internal enthusiasm, and against their

own task-level dashboards is the executive contribution. The review itself is light to install:

- Map one high-volume workflow from cases you observe, not from process documentation alone.
- Label every link as valuable, necessary but reducible, avoidable, or created by the change itself.
- Pilot changes where the tool meets people and systems, and on the exception path, not just on the task.
- Review end-to-end measures and specialist load before expanding volume or authority.

As a planning estimate, not a measured figure, the first review of a substantial workflow takes about two weeks of part-time effort: a dozen traced cases, a few observation sessions, one wall. Run it on a workflow that already has an automation proposed, so the findings land in a live decision. The workflow owner owns the review and carries its cost in staff time. The automation or AI program team supplies the method.

The review also needs a sponsor who is ready for what it finds, because the findings point at design choices that have authors. A review step that catches nothing may have been somebody's governance win. An extra reconciliation step may belong to a team that built it in self-defense. Present the findings as how the process behaves, not as individual fault, and the second review stays welcome. A map that becomes a blame instrument gets no more honest cases to trace.

Timing the measurement window matters as much as picking the measures. The weeks right after an automation lands may be the least representative of its life, because novelty effort looks like work around the tool and learning-curve slowdown looks like permanent cost. As a planning guide, not a measured period, measure the path once the coordination has settled, perhaps one to two quarters in. Then keep the baseline alive, because coordination can grow back around a tool as cases and people change.

Keep one record per workflow: the traced baseline, the links sorted into the four bins with the created-by-the-change bin priced, work around the tool by category and by seniority, handoff queue measures, exception hours against routine savings, and the path-level outcomes behind each decision to scale.

The shift is from counting what a task saves to checking what the whole path gains. Until the path confirms a saving, treat it as a claim to test.

Notes and sources

- [1] Thomas W. Malone and Kevin Crowston, "The Interdisciplinary Study of Coordination," *ACM Computing Surveys* 26, no. 1 (1994): 87–119.
<https://crowston.syr.edu/sites/default/files/acmcs94.pdf>
- [2] Anders Humlum and Emilie Vestergaard, "Still Waters, Rapid Currents: Early Labor Market Transformation Under Generative AI," NBER Working Paper 33777 (May 2025, revised March 2026; first circulated as "Large Language Models, Small Labor Market Effects"). https://www.nber.org/system/files/working_papers/w33777/w33777.pdf
- [3] Rob Cross, Reb Rebele, and Adam Grant, "Collaborative Overload," *Harvard Business Review*, January–February 2016. <https://hbr.org/2016/01/collaborative-overload>
- [4] Erik Brynjolfsson, Daniel Rock, and Chad Syverson, "The Productivity J-Curve: How Intangibles Complement General Purpose Technologies," NBER Working Paper 25148 (2018); published in *American Economic Journal: Macroeconomics* 13(1), 2021. https://www.nber.org/system/files/working_papers/w25148/w25148.pdf
- [5] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," *Advances in Neural Information Processing Systems* 28 (NeurIPS 2015).
- [6] Chintan Amrit, Jos van Hillegersberg, and Kuldeep Kumar, "Identifying Coordination Problems in Software Development: Finding Mismatches between Software and Project Team Structures," 2012.
- [7] Harang Ju, "When Coordination Is Avoidable: A Monotonicity Analysis of Organizational Tasks," 2026 preprint. <https://arxiv.org/abs/2602.18673>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.